GESTION SAV

Fiche de révision — démonstration projet

Application de gestion du service après-vente

Spring Boot 3 · React · PostgreSQL · IA locale (Ollama)

Chiffres clés du projet	
Interventions enregistrées	**33** dont 23 terminées
Utilisateurs	**15** — 1 directeur, 6 techniciens, 8 clients
Chiffre d'affaires cumulé	**3 920 DT**
États du cycle de vie	**7**, de NOUVELLE à TERMINEE ou ANNULEE
Tables en base	**10**, générées par Hibernate
Contrôleurs REST	**7**
Rôles	**3** — Client, Employé SAV, Directeur

À quoi sert ce document

Il rassemble ce qu'il faut savoir réciter le jour de la démonstration : le discours d'ouverture, le scénario minuté, les réponses aux questions techniques probables et la liste des vérifications à faire avant d'entrer en réunion.

Les chiffres ci-dessus proviennent de la base de données réelle et d'appels à l'API testés le 20 septembre 2026.

Les trois vérifications critiques

1. Redémarrer le backend pour activer le correctif de session (code 401).

2. Se connecter aux **trois rôles dans trois fenêtres** le matin même : la double authentification bloquerait la démonstration à chaque changement de rôle.

3. Vérifier qu'Ollama tourne : `curl http://localhost:11434`

1. Le projet en une minute

Le problème. Dans un service après-vente non outillé, les demandes d'intervention arrivent par téléphone ou par courriel. Elles se perdent, personne ne sait qui traite quoi, et la direction n'a aucune visibilité sur l'activité ni sur la charge des techniciens.

La réponse. Une application web qui couvre le cycle complet d'une intervention, du dépôt de la demande par le client jusqu'à l'envoi du rapport de clôture, en passant par l'affectation à un technicien et le suivi du traitement. Chaque acteur dispose de sa propre vue.

Les trois rôles

Rôle	Ce qu'il peut faire
Client	Dépose une demande d'intervention avec photos, suit son avancement en temps réel, dialogue avec l'assistant IA, reçoit le rapport de clôture.
Employé SAV	Consulte les interventions qui lui sont affectées, les accepte, fait évoluer leur état, commente, puis clôture avec un rapport technique et un prix final.
Directeur	Voit toutes les interventions, affecte les techniciens avec l'aide de l'IA, gère les comptes clients et employés, consulte le tableau de bord statistique.

2. Architecture technique

Couche	Technologie	Rôle
Présentation	React + Vite	Interface web sur le port 5173. Appels HTTP via axios.
API	Spring Boot 3 / Java 21	API REST sur le port 8080, préfixe `/api`. Documentée par Swagger.
Sécurité	Spring Security + JWT	Filtre d'authentification, autorisation par rôle sur chaque endpoint.
Persistance	Spring Data JPA / Hibernate	Schéma généré depuis les entités (`ddl-auto: update`).
Données	PostgreSQL	Port 5432, base `sav_db`, 10 tables.
IA	Ollama (qwen2.5:3b)	Modèle exécuté **en local** sur le port 11434.

Le chemin d'une requête

```
Navigateur → axios (+ jeton JWT) → JwtAuthenticationFilter → Controller → Service →
Repository → PostgreSQL
```

Retenez cette chaîne : elle répond à la moitié des questions d'architecture. Chaque couche a une responsabilité unique, et les contrôleurs ne contiennent aucune logique métier.

3. Le cycle de vie d'une intervention

C'est le cœur fonctionnel du projet : sept états et des transitions contrôlées côté serveur. Un technicien ne peut pas clôturer une intervention qu'il n'a pas acceptée.

État	Qui déclenche	Signification
NOUVELLE	Client	Demande créée, en attente d'affectation.
AFFECTEE	Directeur	Un technicien a été désigné.

État	Qui déclenche	Signification
ACCEPTEE	Employé	Le technicien prend la demande en charge.
EN_COURS	Employé	Traitement en cours.
EN_ATTENTE	Employé	Bloquée : attente de pièces ou de validation.
TERMINEE	Employé	Clôturée avec rapport technique et prix final.
ANNULEE	Directeur	Demande abandonnée.

Chaque transition est horodatée et enregistrée dans la table `historique_intervention` : on sait toujours qui a fait quoi et quand. C'est l'argument à sortir si l'on vous interroge sur la traçabilité.

4. La sécurité — à maîtriser absolument

C'est le sujet sur lequel une encadrante creuse le plus. Voici le mécanisme complet, dans l'ordre.

L'authentification en trois étapes

- **Étape 1.** `POST /auth/connexion` avec courriel et mot de passe. Le mot de passe est comparé à son empreinte **BCrypt** stockée en base. Aucun jeton n'est encore délivré : la réponse indique seulement `twoFactorRequired: true`.
- **Étape 2.** L'utilisateur saisit le code à six chiffres de son application d'authentification (norme **TOTP**, code valable 30 secondes), ou demande un code par courriel.
- **Étape 3.** `POST /auth/2fa/verify`. Le code vérifié, le serveur délivre un **jeton JWT signé en HS256**, valable 24 heures. Le navigateur le conserve et l'envoie dans l'en-tête `Authorization: Bearer ...` à chaque appel.

L'autorisation

Le jeton prouve *qui* vous êtes ; l'annotation `@PreAuthorize` posée sur chaque méthode de contrôleur décide *ce que* vous avez le droit de faire. Lister toutes les interventions exige par exemple le rôle Directeur.

Les autres protections

- **CORS restreint** : seules les origines déclarées peuvent appeler l'API.
- **Sessions sans état** : le serveur ne mémorise rien, tout est dans le jeton. L'API est donc extensible horizontalement.
- **Secrets externalisés** : mot de passe de base, clé JWT et identifiants de messagerie passent par des variables d'environnement.

> **Anecdote à raconter — elle fait toujours bonne impression**
>
> « J'ai identifié et corrigé un défaut récemment. Quand le jeton expirait, Spring Security répondait **403 Interdit** au lieu de **401 Non authentifié**, faute de point d'entrée d'authentification configuré. Côté client, l'intercepteur ne réagissait qu'au 401 : l'utilisateur restait donc bloqué avec un jeton mort et un message d'erreur générique, sans jamais être invité à se reconnecter. J'ai ajouté un `authenticationEntryPoint` qui renvoie bien 401, et différencié les messages d'erreur côté interface. »
>
> **Pourquoi c'est efficace :** cela montre que vous savez diagnostiquer, distinguer 401 et 403, et raisonner sur l'expérience utilisateur, pas seulement coder.

5. Les deux fonctions d'intelligence artificielle

Le modèle tourne **en local** via Ollama. Trois avantages à mettre en avant : gratuité totale, confidentialité des données clients qui ne quittent jamais la machine, et absence de dépendance à un service externe.

a) L'assistant conversationnel

Le visiteur décrit sa panne. L'IA propose des pistes de dépannage et, si le problème persiste, propose de créer une demande d'intervention **pré-remplie** : elle déduit la description et le niveau de priorité de la conversation. Temps de réponse mesuré : environ deux secondes.

b) L'affectation intelligente

C'est la fonction la plus convaincante en démonstration. Au moment d'affecter une intervention, le système classe les techniciens selon **leur charge active** et **leur historique d'interventions terminées**, puis justifie sa recommandation en langage naturel.

Exemple de sortie réelle : *« 0 intervention(s) en cours et 8 déjà traitée(s) »*. Tous les candidats restent affichés avec leurs indicateurs.

> **La phrase qui fait la différence**
>
> « L'IA assiste le directeur, elle ne décide pas à sa place. La recommandation est justifiée et tous les candidats restent visibles : la décision finale reste humaine. »
>
> Cette formulation montre une maturité de conception — vous avez réfléchi à la place de l'automatisation, pas seulement à sa prouesse technique.

6. Le scénario de démonstration (20 minutes)

Le principe : **une seule intervention suivie du début à la fin**, vue successivement par les trois rôles. Un fil narratif unique retient bien mieux l'attention qu'une visite de menus.

Temps	Étape	Ce que vous faites et dites
2 min	**Ouverture**	Annoncez le problème métier, puis la pile technique. Montrez les diagrammes **dix secondes**, pas plus : on veut voir l'application tourner.
2 min	**Le chatbot public**	Non connecté, posez une vraie question de panne. Soulignez que l'IA est locale et qu'elle pré-remplit la demande.
3 min	**Le client dépose**	Créez une intervention et **joignez une photo**. Montrez le code unique généré et le suivi côté client.
4 min	**L'affectation IA**	**Votre meilleur moment : prenez votre temps.** Lisez la justification à voix haute, montrez les candidats classés, puis affectez.
4 min	**Le technicien traite**	Acceptez, passez en cours, commentez, clôturez avec rapport et prix. Mentionnez l'envoi du rapport par courriel sans forcément le déclencher.
3 min	**Le pilotage**	Tableau de bord : répartitions, chiffre d'affaires, performance par technicien. Citez les chiffres réels.
2 min	**Clôture**	Base de connaissances, puis annoncez **vous-même** les limites et la suite du travail.

Terminer en annonçant vos propres limites vous protège : ce qui est présenté comme une perspective assumée ne peut plus vous être reproché comme un oubli.

7. Les questions probables et vos réponses

Question	Réponse
Pourquoi une IA locale plutôt que ChatGPT ?	Gratuité, confidentialité des données clients, aucune dépendance réseau ni clé d'API à gérer.
Comment sécurisez-vous l'API ?	JWT signé HS256, double authentification TOTP, `@PreAuthorize` par rôle sur chaque endpoint, CORS restreint, mots de passe en BCrypt.
Que se passe-t-il si le jeton expire ?	Le serveur répond 401, l'intercepteur détecte la session expirée et redirige vers la connexion. *Enchaînez sur l'anecdote de la section 4.*
Qui a créé le schéma de base ?	Hibernate le génère à partir des entités JPA. Dix tables, aucun script SQL manuel.
Avez-vous écrit des tests ?	**Répondez honnêtement.** Si vous n'en avez pas, dites que c'est la priorité suivante et citez ce que vous testeriez d'abord : les transitions d'état et la sécurité. Ne bluffez jamais.
Est-ce déployable en production ?	Oui : toute la configuration passe par variables d'environnement. Il faut fournir une clé JWT, une base et des identifiants de messagerie externes.
Pourquoi React plutôt qu'un rendu serveur ?	Séparation nette entre API et interface : l'API peut servir d'autres clients, dont une application mobile.
Comment gérez-vous les pièces jointes ?	Téléversement multipart, types autorisés restreints aux PDF et images, taille limitée à 10 Mo.

8. Les trois pièges qui peuvent faire dérailler la démo

Risque	Parade
La double authentification Tous les comptes utiles l'ont activée. Chaque changement de rôle exigerait un code.	Le matin même, connectez-vous aux **trois rôles dans trois fenêtres distinctes** (une normale et deux privées). Vous basculez ensuite par Alt+Tab. Les jetons durent 24 heures.
Les données de test Des noms comme « Test NonAffecte » ou « Emp RaceB » sont visibles dans les listes *et dans la fenêtre de suggestion IA*.	Renommez ces comptes avec des noms crédibles avant la réunion. C'est l'amélioration la plus rentable pour l'impression générale.
Les graphiques plats 32 interventions sur juillet et une seule sur septembre : la courbe d'évolution sera un pic isolé.	Soit vous étalez quelques dates en base, soit vous devancez la remarque : « les données sont des jeux de test concentrés sur ma période de développement ».

9. Liste de vérification

La veille

- [ ] Redémarrer le backend pour activer le correctif de session.
- [ ] Tester l'envoi du rapport de clôture par courriel.
- [ ] Nettoyer ou assumer les noms de comptes de test.
- [ ] Tester le chatbot deux ou trois fois et retenir la meilleure formulation.
- [ ] **Faire une répétition complète du scénario, chronomètre en main.**

Le jour même, avant d'entrer

- [ ] Lancer `start.bat` et attendre le démarrage complet du backend.
- [ ] Vérifier Ollama : `curl http://localhost:11434` doit répondre.
- [ ] Se connecter aux trois rôles dans trois fenêtres et les laisser ouvertes.
- [ ] Ouvrir les diagrammes dans des onglets déjà chargés.
- [ ] Préparer une intervention en état NOUVELLE pour la démonstration d'affectation.
- [ ] Désactiver les notifications du système et passer le téléphone en silencieux.

Le conseil le plus important

Faites **une répétition complète à voix haute**, chronomètre en main, avant la réunion.

Une démonstration répétée une seule fois se passe toujours mieux qu'une démonstration improvisée, même quand on connaît son code par cœur. Ce n'est pas le code qui vous trahira, c'est l'enchaînement des fenêtres et le fil du discours.

Repères techniques à avoir en tête

Élément	Valeur
Interface web	`http://localhost:5173`
API REST	`http://localhost:8080/api`
Documentation Swagger	`http://localhost:8080/api/swagger-ui.html`
Base de données	PostgreSQL, port 5432, base `sav_db`
Moteur d'IA	Ollama, port 11434, modèle `qwen2.5:3b`
Validité du jeton	24 heures

Bonne chance. Vous connaissez ce projet mieux que quiconque dans la salle : la démonstration ne fait que le rendre visible.